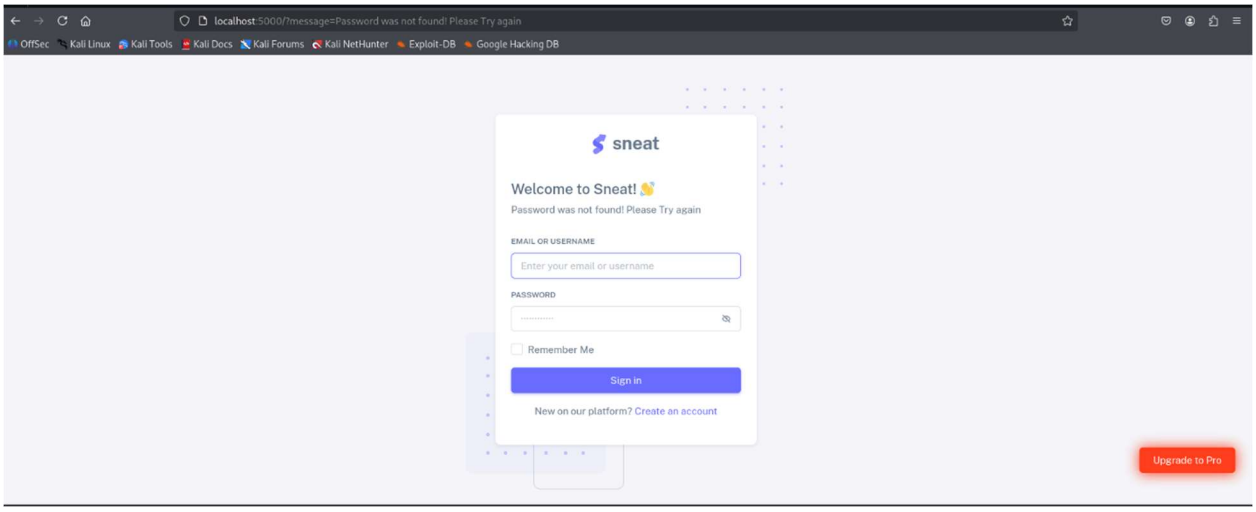
Cybersecurity Internship

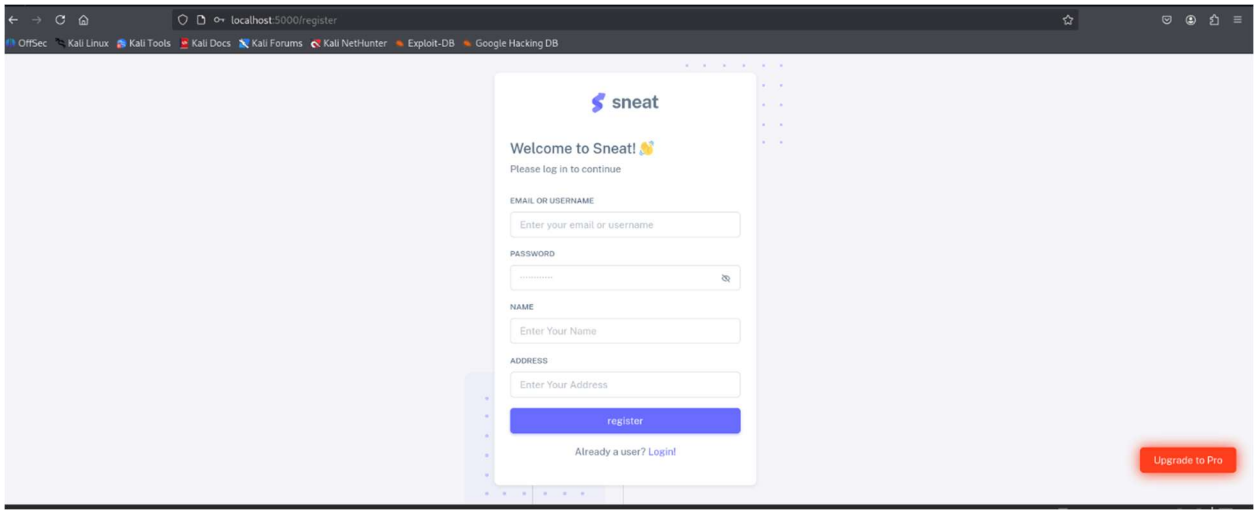
Strengthening Security Measures for a Web Application

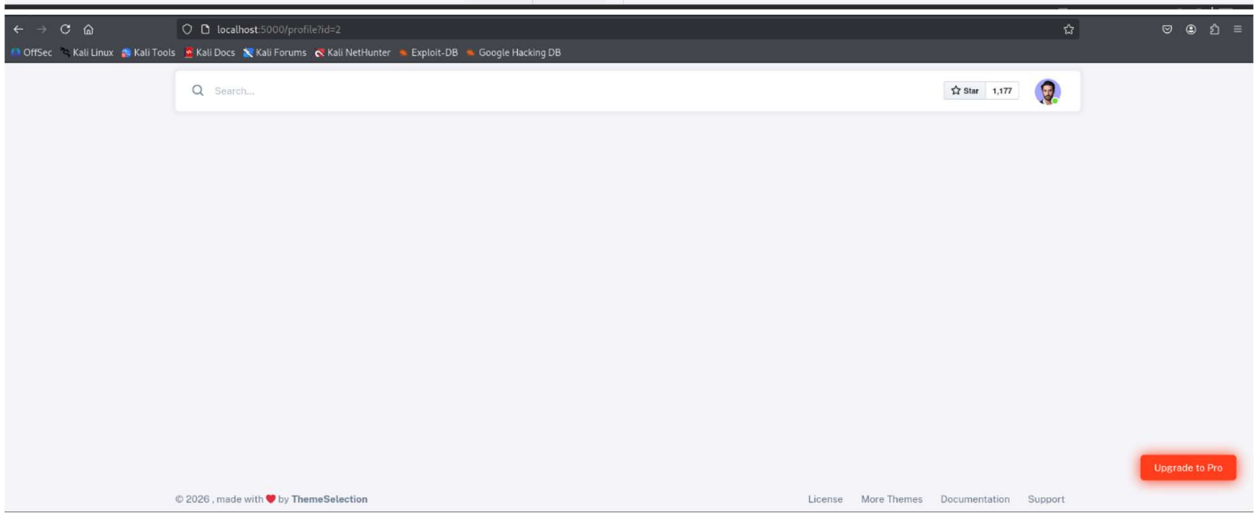
Week 1: Security Assessment

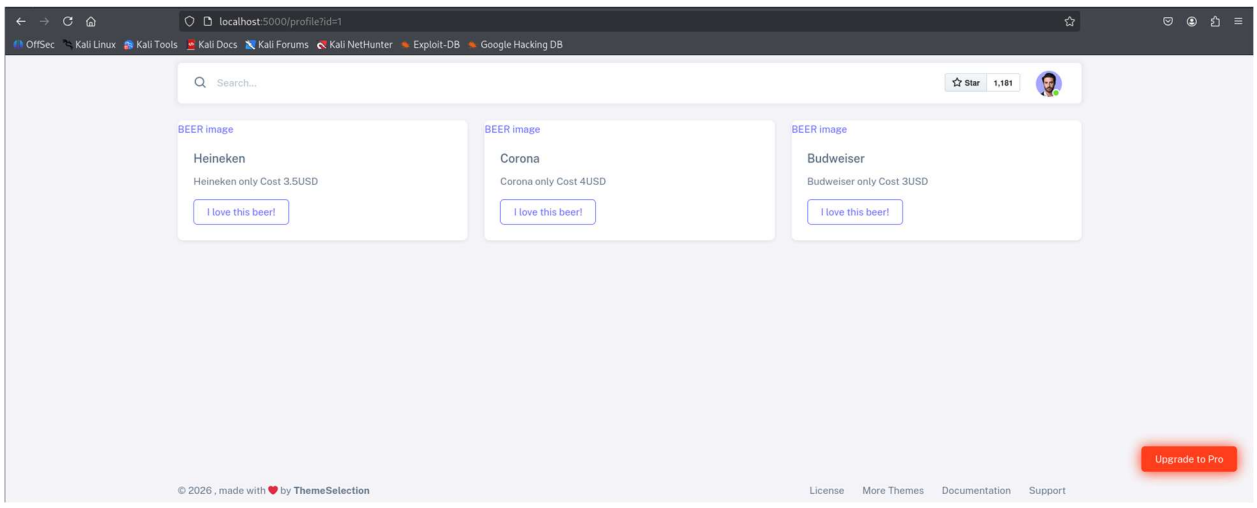
1. GitHub clone:

2. Website pages:

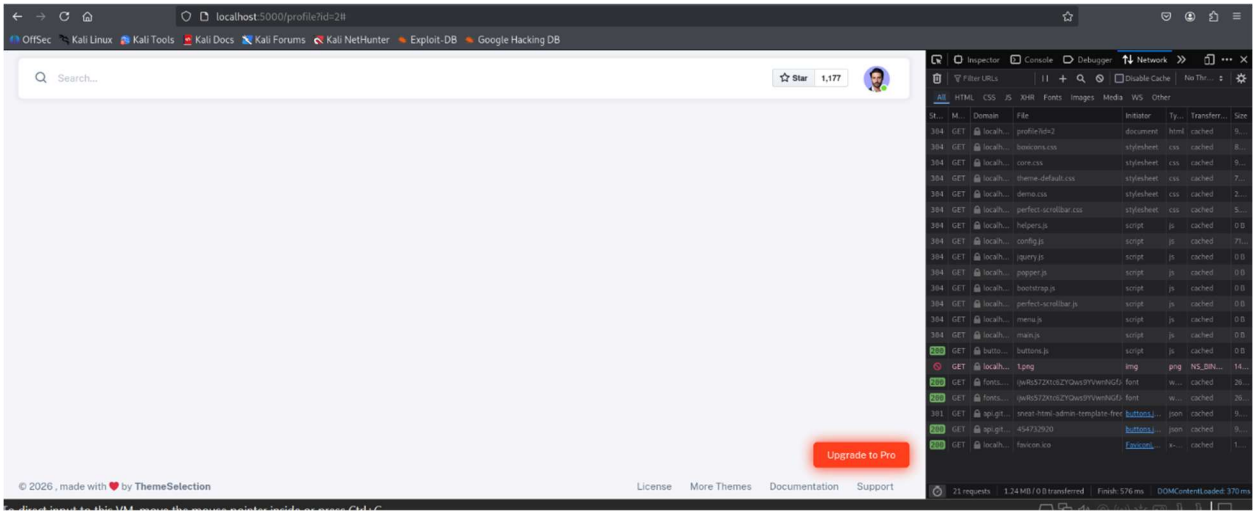



localhost:5000/register
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
sneat
Welcome to Sneat!
Please log in to continue
EMAIL OR USERNAME
Enter your email or username
PASSWORD
NAME
Enter Your Name
ADDRESS
Enter Your Address
register
Already a user? Login!
Upgrade to Pro


localhost:5000/profile?id=2
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Search...
Star 1,177
Upgrade to Pro
© 2026 , made with by ThemeSelection
License More Themes Documentation Support

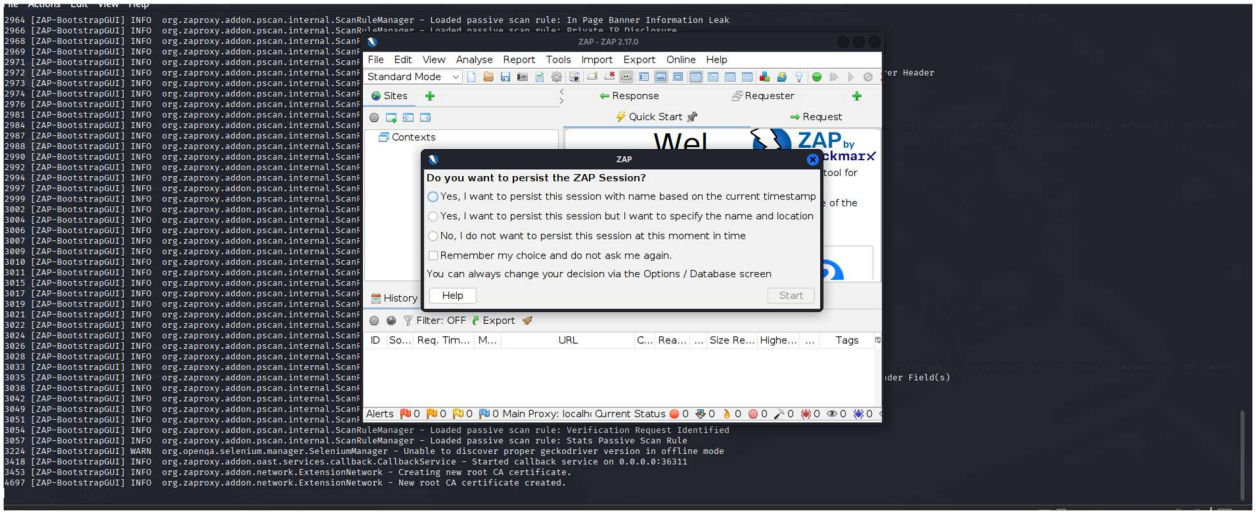

localhost:5000/profile?id=1
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Search...
Star 1,181
BEER image
Heineken
Heineken only Cost 3.5USD
I love this beer!
BEER image
Corona
Corona only Cost 4USD
I love this beer!
BEER image
Budweiser
Budweiser only Cost 3USD
I love this beer!
Upgrade to Pro
© 2026 , made with by ThemeSelection
License More Themes Documentation Support

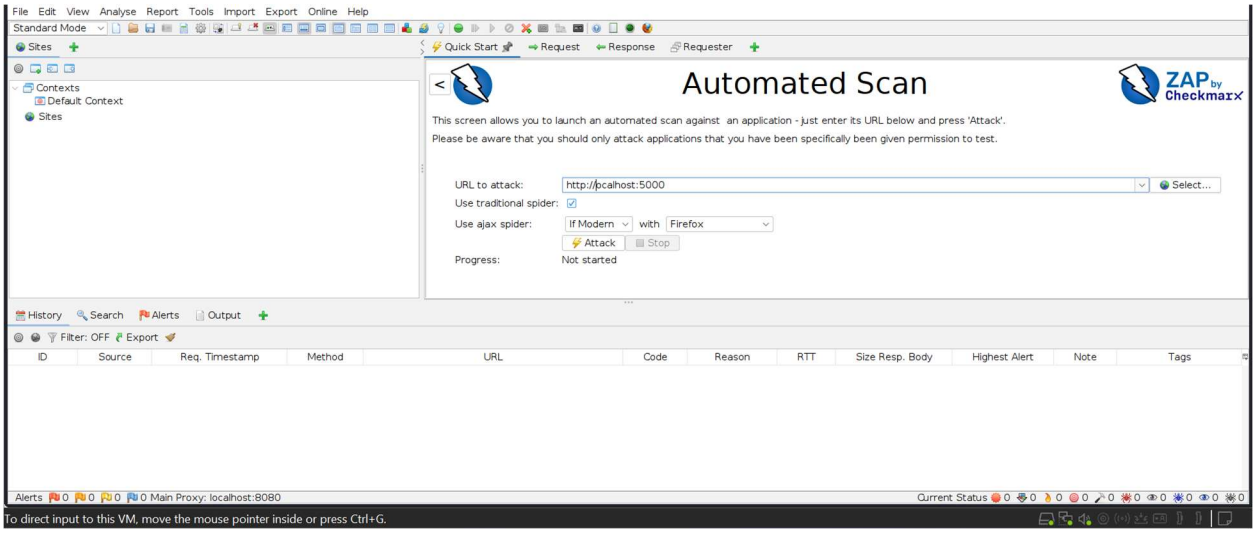
3. Check the signup, login, and profile pages.

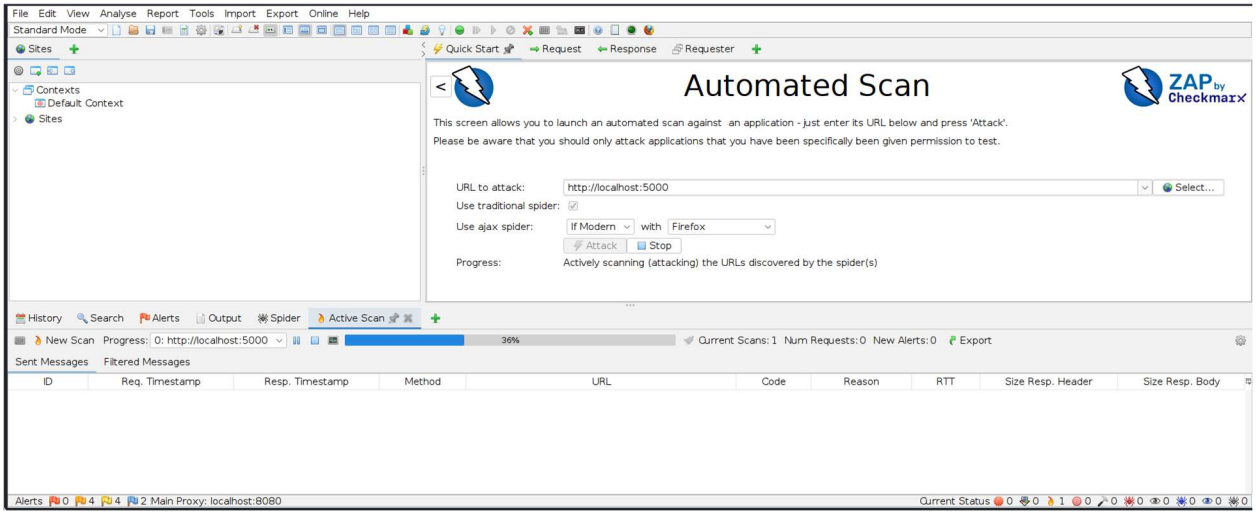


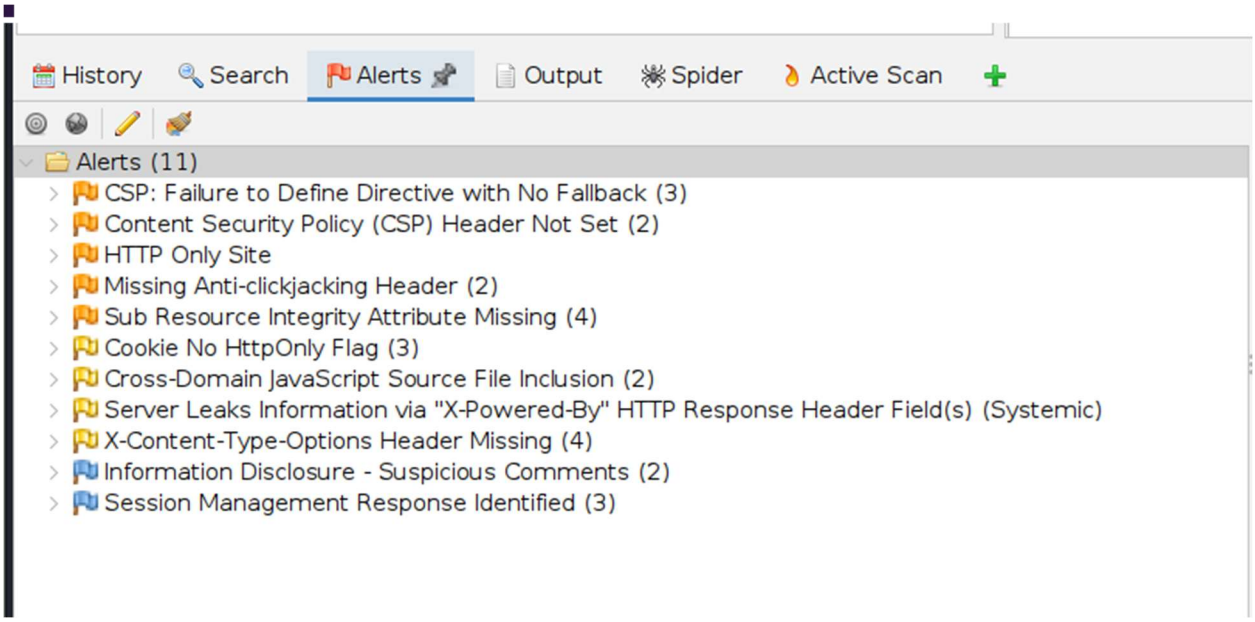
I explored the application's Dev Tools

4. OWASP ZAP:



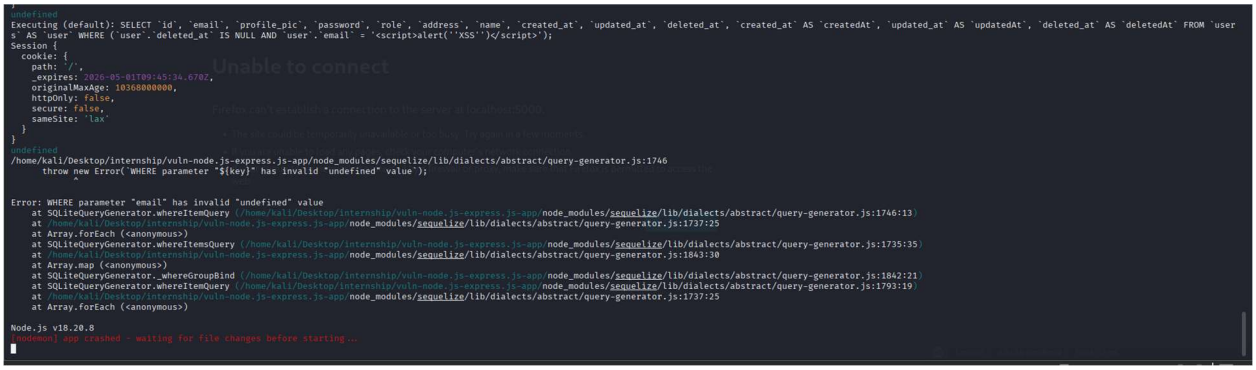






The OWASP ZAP automated scan identified multiple security misconfigurations and session management weaknesses. Combined with manual testing results, these findings confirm that the application is vulnerable to XSS, session hijacking, and information disclosure attacks. The complete report is attached.

5. Browser Developer Tools: XSS attacks

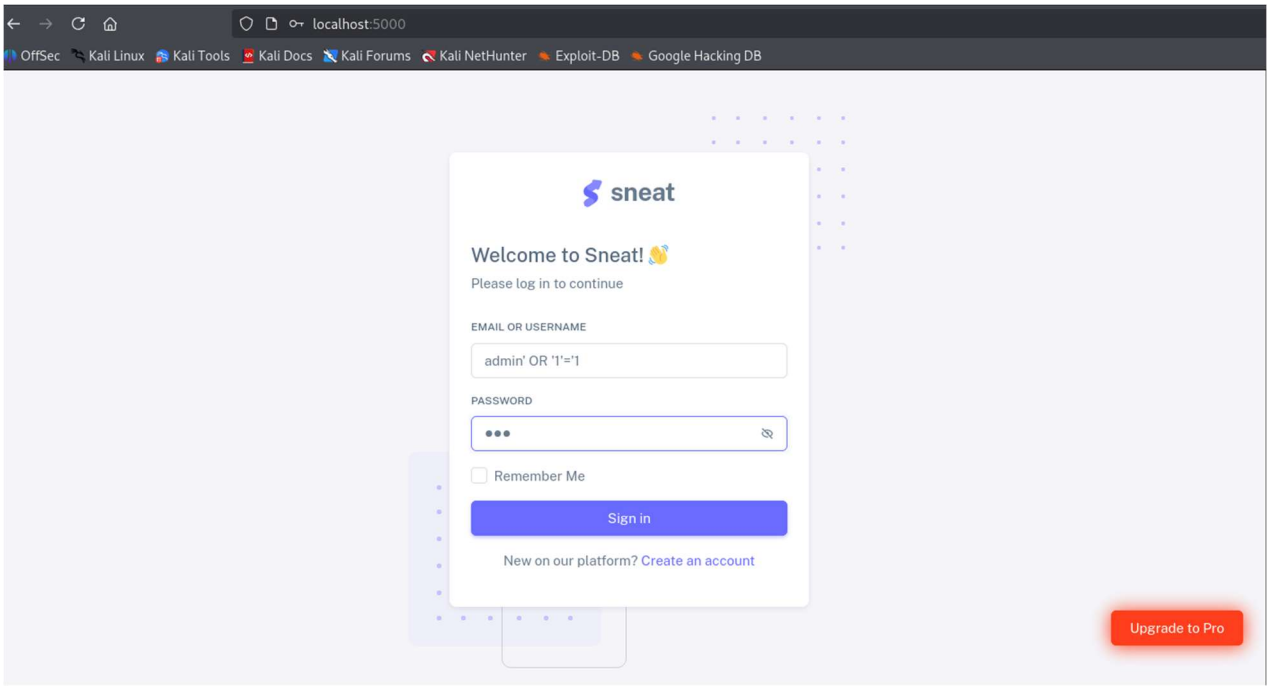


The application does not validate user input before processing it in backend database queries. Malicious or malformed input can cause backend variables to become undefined, resulting in unhandled exceptions. The following payload caused the application to crash:

<script>alert('XSS')</script>

This resulted in the error WHERE parameter "email" has invalid "undefined" value. The Node.js server terminated, causing a denial of service.

6. Basic SQL Injection Test



Attempted authentication bypass using SQL injection payload:

admin' OR '1'='1

The application did not authenticate the user. The ORM (Sequelize) escaped the payload, preventing SQL execution. The application is not vulnerable to classic SQL Injection attacks. However, the application does not validate or reject malicious input, which may lead to other injection-based attacks such as XSS.

Week 2: Implementing Security Measures

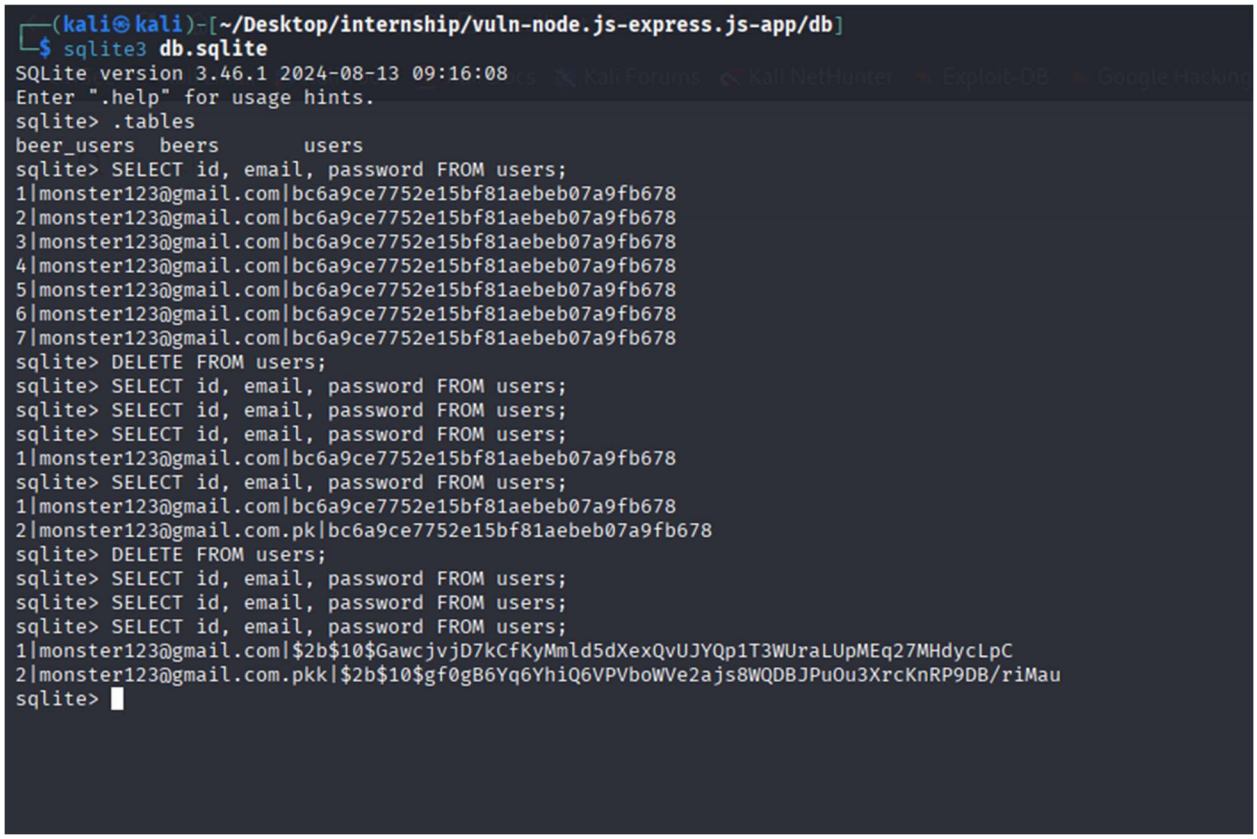
1. **Fix Vulnerabilities**

- Sanitize and Validate Inputs: validator

```
 */
app.post('/v1/user/', async (req, res) => {

//const userEmail = req.body.email || '';
const userEmail = validator.normalizeEmail(req.body.email || '');
//const userName = req.body.name || '';
const userName = validator.escape(req.body.name || '');
const userRole = req.body.role || 'user';
//const userPassword = req.body.password || '';
const userPassword = validator.escape(req.body.password || '');
const userAddress = validator.escape(req.body.address || '');

// Email validation (basic + crash safe)
if (!validator.isEmail(userEmail)) {
    return res.status(400).json({ error: 'Invalid email format' });
}

// Password length check
if (userPassword.length < 6) {
    return res.status(400).json({ error: 'Password must be at least 6 characters' });
}

try {
    // Hash password
    const hashedPassword = await bcrypt.hash(userPassword, 10);

    const new_user = await db.user.create({
        name: userName,
        email: userEmail,
        role: userRole,
```

The validator library was implemented to validate and sanitize user input, especially email fields and query parameters. Invalid input is now rejected before being processed by the application, preventing crashes and injection attacks.

- Password Hashing: Use bcrypt to hash



Passwords were secured using bcrypt. User passwords are now hashed and salted before being stored in the database, ensuring that plaintext credentials are never saved. The passwords were firstly stored in MD5 which was changed to bcrypt.

2. Enhance Authentication

```
module.exports = (app,db) => {


function verifyToken(req, res, next) {

    const token = req.session.token;

    if (!token) {
        return res.redirect('/?message=Authentication required');
    }

    try {
        const decoded = jwt.verify(token, JWT_SECRET);
        req.user = decoded; // attach user info
        next();
    } catch (err) {
        return res.redirect('/?message=Invalid or expired token');
    }
}

```

```
     * @param {string} profile_description
     */
    app.get('/profile', verifyToken, (req,res) =>{
```

Basic token-based authentication was implemented using JSON Web Tokens (JWT). After successful login, a signed token is issued to the user and used for authentication in protected routes. This improved session handling and reduced reliance on insecure session mechanisms.

3. Secure Data Transmission

```
'user strict';
const helmet = require('helmet');
const express = require('express'),
    //morgan = require('morgan'),
    config = require('./config'),
    router = require('./router'),
    bodyParser = require('body-parser'),
    db = require('./orm')




const sjs = require('sequelize-json-schema');


const app = express()
const PORT = config.PORT;
//OPTIONAL: Security headers?????
// app.use((req, res, next) => {
//     res.header('Content-Type', 'application/json');
//     res.header("Access-Control-Allow-Origin", "*");
//     res.header("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE");
//     res.header("Access-Control-Allow-Headers", "Origin, X-Requested-With, Content-Type, Accept");
//     next();
//   });

//OPTIONAL: Activate Logging
// app.use(morgan('combined'));
app.use(helmet());
app.use(bodyParser.json());


//session middleware
var cookieParser = require('cookie-parser');
```

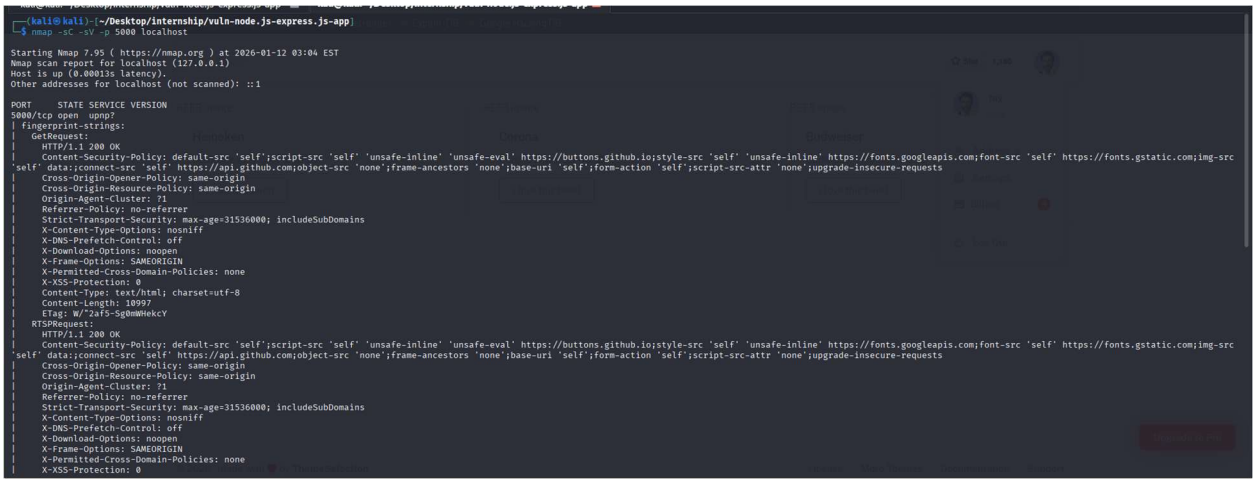
```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~/Desktop/internship/vuln-node.js-express.js-app]
└─$ curl -X POST http://localhost:5000/v1/user \
-d '{"email":"aaa@", "password":"123"}'

{"error":"Invalid email format"}
```

Although the application runs on HTTP in a localhost environment, HTTPS readiness was verified. Security headers such as HSTS were enabled using Helmet. This ensures the application can be deployed securely behind a TLS-enabled proxy in production environments.

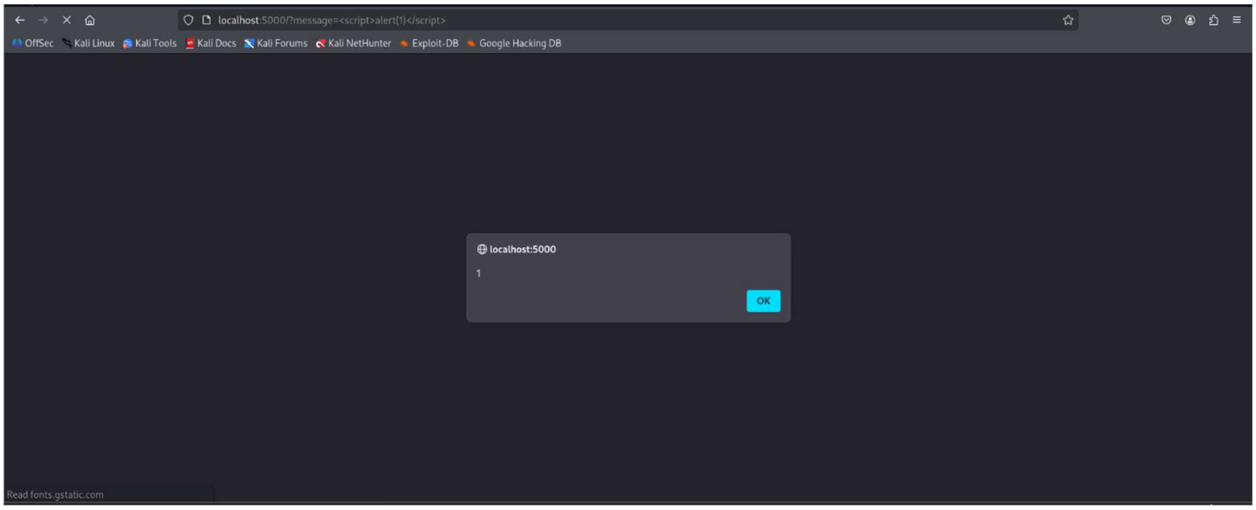
Week 3: Advanced Security and Final Reporting

1. Basic Penetration Testing

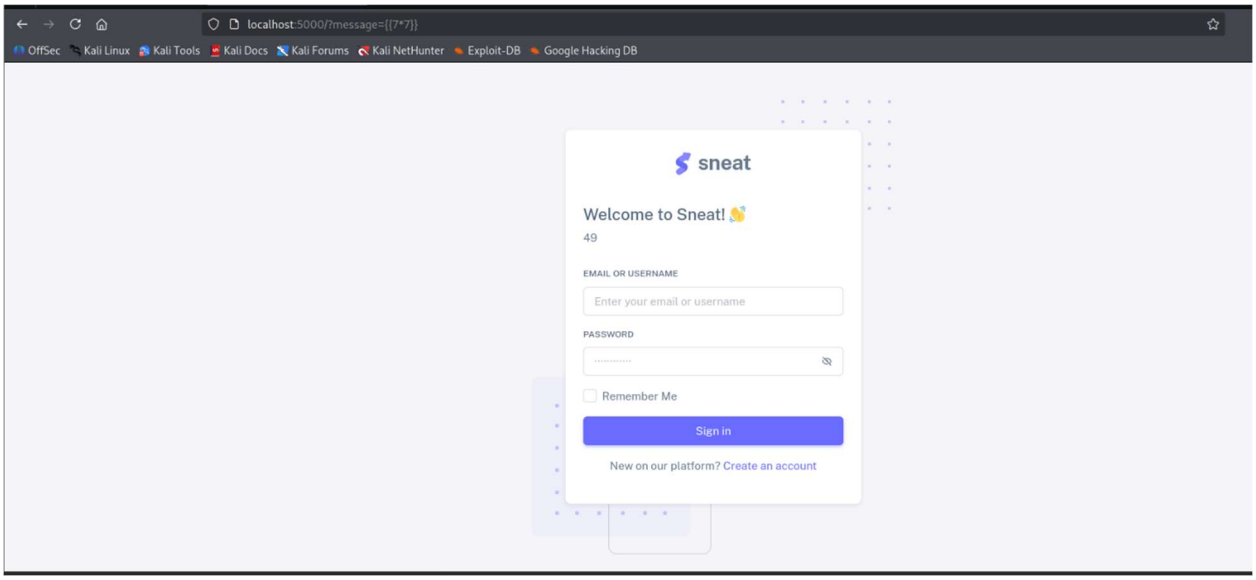
- Use tools like Nmap or browser-based testing to simulate common attacks.



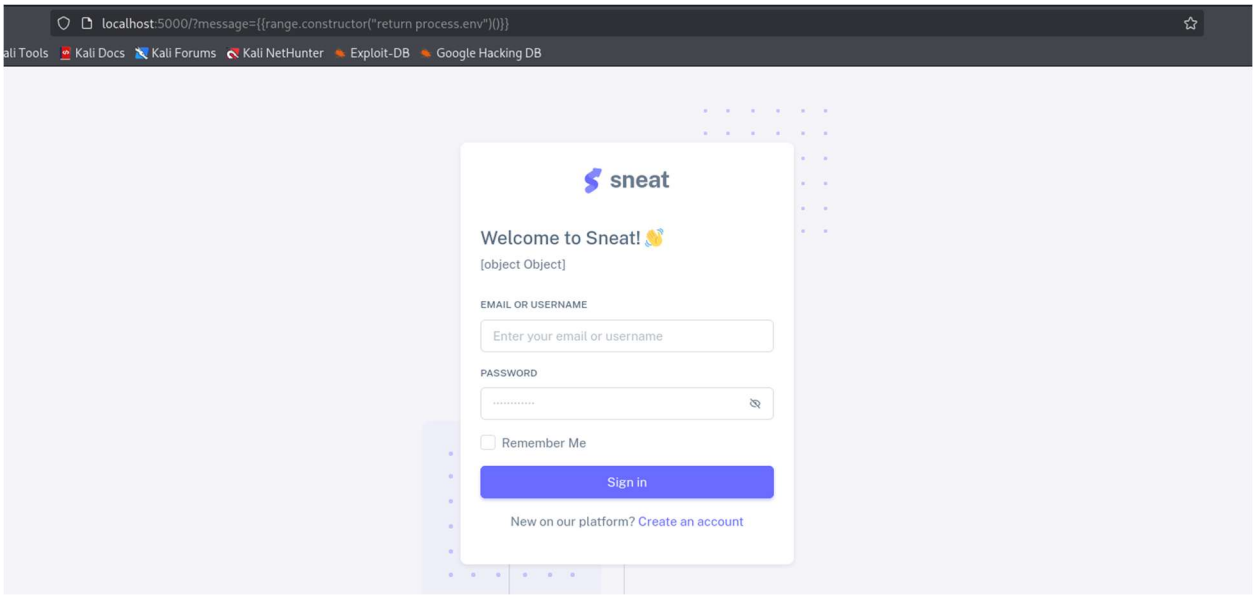
An Nmap scan was conducted using default scripts and version detection. The application exposed only one TCP port (5000), which minimized the attack surface. Service fingerprinting was inconclusive, which is typical for custom Node.js applications. Security headers such as CSP, HSTS, and X-Frame-Options were detected, confirming successful implementation of browser-level protections.



The above demonstrates a Reflected Cross-Site Scripting (XSS) attack on the application. By supplying the payload <script>alert(1)</script> through the message URL parameter, the application rendered the input directly into the response without proper output encoding. This caused the browser to execute the injected JavaScript and display an alert dialog, confirming that arbitrary client-side code can be executed in the context of the user's session.

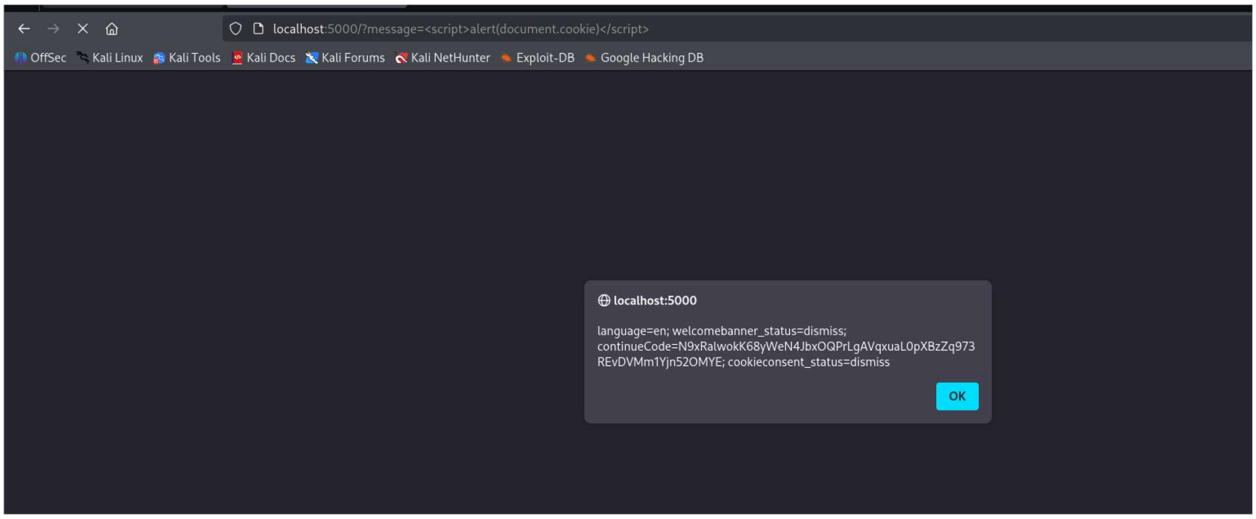


In this the payload {{7*7}} was evaluated by the application and rendered as 49, indicating the presence of a Server-Side Template Injection (SSTI) vulnerability.

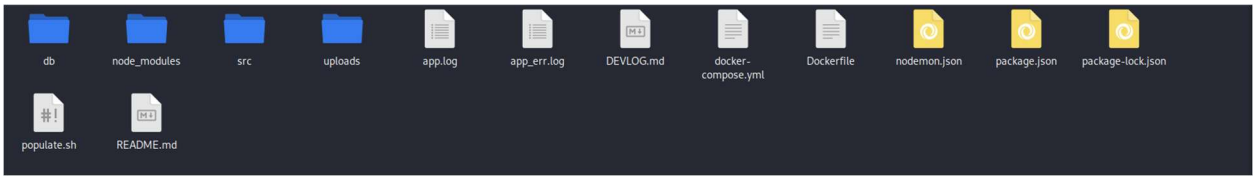


This demonstrates a more advanced Server-Side Template Injection (SSTI) payload using {{range.constructor("return process.env")()}}, which resulted in the output [object Object].

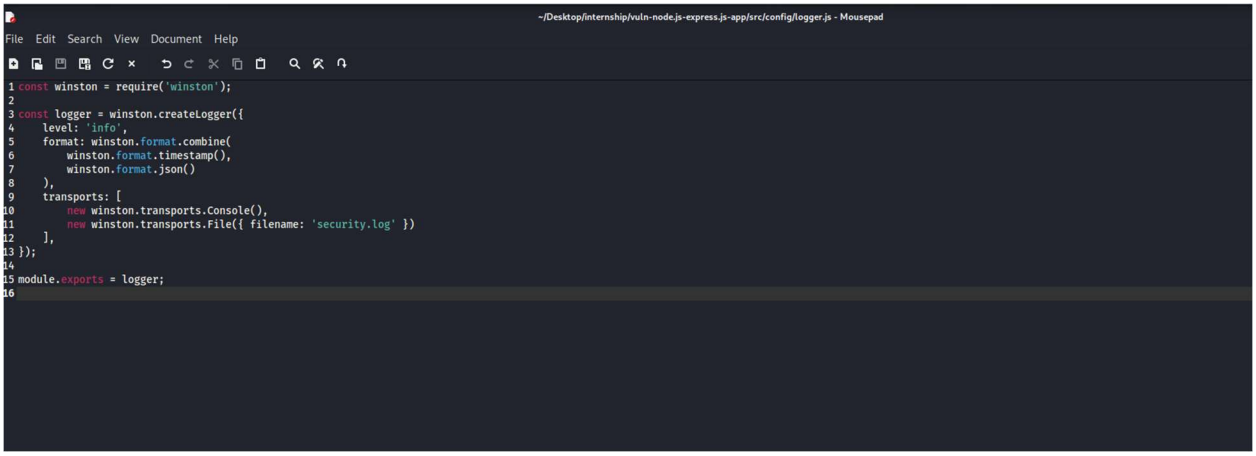
This represents a high-risk vulnerability, as it may lead to sensitive information disclosure and, in some cases, remote code execution if the attacker is able to further manipulate the execution context.

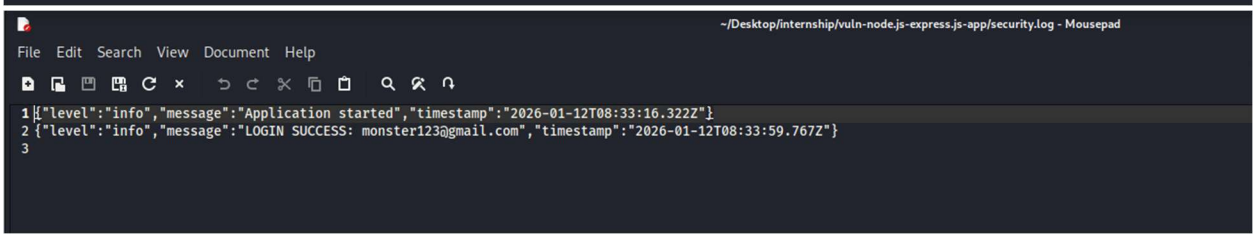


2. Set Up Basic Logging



The application already had a logging system setup, but as required another similar one was setup using Winston.





3. Create a Simple Checklist

1. Application-Level Security

- Input validation using the validator library
- Proper handling of undefined or malformed inputs to prevent crashes
- Improved error handling to avoid server crashes (DoS prevention)

2. Authentication & Authorization

- Password hashing and salting using bcrypt
- Secure password storage
- JSON Web Token (JWT)–based authentication
- Protection of sensitive routes using authentication middleware

3. Session & Client-Side Security

- Inspection and hardening of cookies and session data
- Reduced exposure of sensitive identifiers in client-side code

4. Transport & HTTP Security

- HTTPS readiness verified for production deployment
- Security headers implemented using Helmet:
    - Content Security Policy (CSP)
    - HTTP Strict Transport Security (HSTS)
    - X-Frame-Options
    - X-Content-Type-Options